STALENESS-AWARE PREDICTIVE LOAD BALANCER

SA-PLB — Complete Project Learning Guide

A practical, step-by-step guide from the base research idea to our dataset, model training, staleness layer, experiments, results, commands and viva questions.

Important accuracy note: This guide uses project details and recorded results available in our discussion. The base-paper model details were recorded in our earlier discussion as CatBoost; the original paper PDF is not attached here, so verify those exact details against the paper before formal submission.

How to use this guide

Read in order. Chapters follow the order in which the project was built. “Say this in evaluation” paragraphs are ready-to-practise explanations.

Project at a glance

Item	Our project
Name	Staleness-Aware Predictive Load Balancer (SA-PLB)
Problem	ML routing may make poor choices when monitoring inputs are outdated.
Approach	Check observation age; use ML if fresh and WRR if stale.
Policies	RR, WRR, ML-based PLB and SA-PLB.
Environment	Python discrete-event simulation with three simulated replicas.
Model	Scikit-learn HistGradientBoostingRegressor.
Contribution	A freshness-aware routing gate with WRR fallback; not a newly invented ML algorithm.

One-sentence summary

> “We built a simulated load balancer that predicts response time, but trusts the ML prediction only when its monitoring inputs are fresh; otherwise it falls back to Weighted Round Robin.”

Contents

1. Load balancing basics
2. Base research paper
3. Research gap and contribution
4. Four routing policies
5. Dataset and exact feature meanings
6. Simulation and monitoring data
7. ML model and model choice
8. Training and evaluation
9. Staleness induction and freshness logic
10. Experiments and commands
11. Results and interpretation
12. Limitations
13. Evaluation questions
14. Glossary and final explanation

1. Load balancing from zero

A client (browser, app or another service) sends a request. A backend server/container processes it and returns a response. If all requests go to one backend, it can become overloaded while other backends are idle. A **load balancer** sits between clients and backend replicas and chooses where each request goes.

A **replica** is one running instance/copy of the service. Our simulation has three replicas: R0, R1 and R2. **Response time** is the time taken for a request to complete, measured in milliseconds (ms). Lower response time is generally better.

Why this matters

A load balancer should distribute work in a way that avoids unnecessary overload and reduces request delay. But different routing policies make decisions using different information: simple rotation, configured capacity, or ML predictions.

Say this in evaluation: “Load balancing distributes incoming requests across backend replicas. Our objective is to study whether predictive routing can be made more reliable when monitoring data is delayed.”

2. Base research paper: its idea

The base paper discussed for this project is titled *Predictive Load Balancing in Distributed Systems: A Comparative Study of Round Robin, Weighted Round Robin, and a Machine Learning Approach*. Its central idea is to compare fixed routing rules with an ML approach that predicts response latency for candidate backends.

The three approaches

Round Robin (RR): send requests to backends in a repeating rotation. It is simple, but does not consider differences in current load or capacity.
Weighted Round Robin (WRR): assign each backend a weight; higher-weight backends receive a larger share. It reflects configured capacity, but weights may not reflect sudden live changes.
ML approach: estimate response latency for each candidate backend and choose the one with the smallest predicted latency.

What model did the base paper use?

The details recorded in our earlier discussion identify the paper’s model as **CatBoost regression**. The notes describe 20,000 Round-Robin training samples; features of HTTP method, URL, candidate CPU cores, candidate memory, active requests and average latency of the last 50 requests; and actual response latency as the target. The described method predicts latency for each candidate container and chooses the minimum.

Verification caution: the original paper PDF is not attached in this chat. Verify the model name, 20,000 samples and exact feature list against the original source before quoting them as confirmed paper facts. Do not invent its hyperparameters, split or exact results.

What does regression mean?

Regression predicts a numerical value. Here the predicted number is response latency in milliseconds. Classification instead predicts a category, such as fresh/stale.

Say this in evaluation: “The base paper compares RR, WRR and an ML latency predictor. The ML approach estimates response latency on candidate containers and chooses the lowest prediction. We focus on an additional risk: those predictions may be based on stale monitoring information.”

3. Research gap and our contribution

A monitoring record may have been accurate when collected but outdated by the time the load balancer uses it. Example: at 12.0 seconds, R1 has a short queue. The monitor collects that value. New requests then build up the queue, but a delayed monitoring update means the load balancer still sees the old low-queue value. The ML model may therefore underestimate R1's response time.

Our research question

> "Can we make ML-based routing more robust by checking the age of monitoring observations and falling back to a non-ML policy when those observations are too old?"

Our contribution is a **decision layer around an existing ML predictor**. It checks monitoring freshness and chooses between ML routing and WRR. We did not invent a new gradient-boosting algorithm.

4. Four policies in our project

Policy	How it chooses	Information used
RR	Cycles through R0, R1, R2.	Rotation order
WRR	Distributes according to configured weights.	Replica capacity weights
ML / PLB	Predicts response time per candidate; chooses the lowest.	Monitoring + request + replica features
SA-PLB	ML if observations are fresh; otherwise WRR.	ML prediction, freshness gate and fallback

Our three simulated replicas

Replica	CPU cores	Relative speed	Memory
R0	2	1.0	1024 MB
R1	2	0.9	1024 MB
R2	1	1.1	512 MB

WRR weights are calculated as round(2 × cores × speed), giving [4, 4, 2], or a 40%, 40%, 20% share over many requests. These are configured weights, not live CPU measurements.

Why compare all four?

RR is the simple baseline; WRR is capacity-weighted; ML shows prediction without a freshness check; SA-PLB tests the added freshness gate and fallback. Within each experiment run, policies use the same generated workload trace and monitoring schedule, making the comparison paired and fairer.

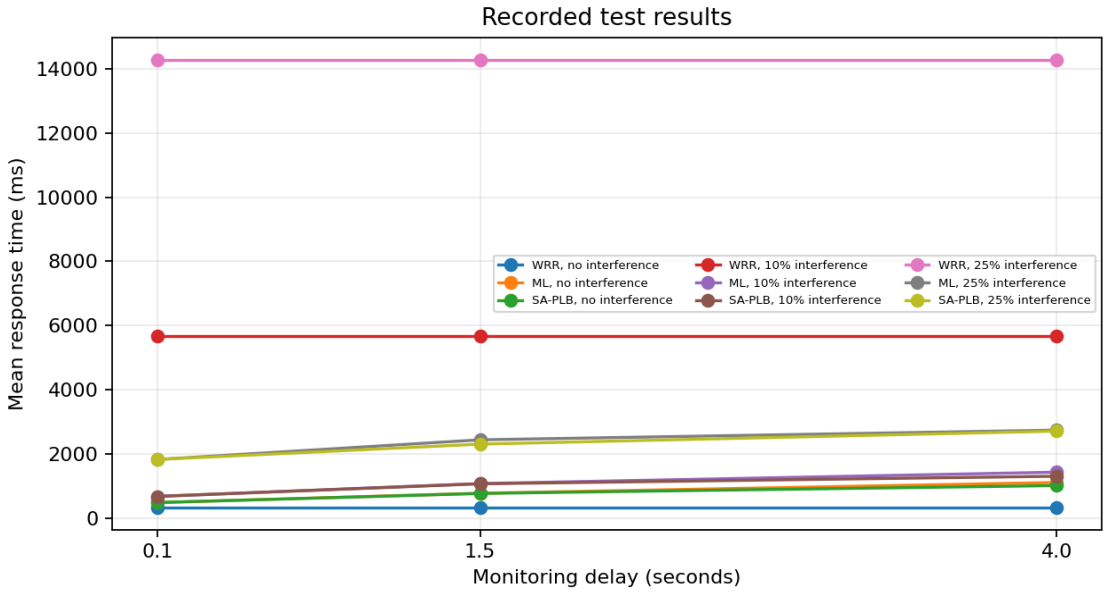
5. Dataset: what did we actually use?

The final experiment pipeline uses a **synthetically generated dataset** created by our Python simulation. It is not a downloaded production dataset. The simulator creates request arrivals, request sizes/types, replica states, monitoring observations and simulated response-time labels.

Important: an earlier project stage also discussed *hybrid_dataset.csv*, described as 16,347 rows and 16 columns, with fields including timestamp, decision_time, request_id, container_id, task_size_kb, request_type, active_requests, avg_latency_ms, cpu_pct, mem_pct, metric_collected_at, staleness_s, true_active_requests, policy, phase and response_time_ms. This is an earlier artifact. Do not mix its row count/schema with the final training log described below.

Final model training split

Subset	Rows	Purpose
Training	54,749	Fit model
Validation	9,962	Select model settings
Testing	8,700	Evaluate on held-out examples



The final split is period-based: earlier periods are used for training, period 6 for validation and period 7 for testing. The three subsets total 73,411 rows.

Exact final model input features

Feature name	Meaning
obs_active	Number of active requests observed on that replica
obs_queue	Number of requests waiting in its queue
obs_cpu_pct	Observed CPU utilisation percentage
obs_mem_pct	Observed memory utilisation percentage
obs_recent_rt_ms	Recently observed response time, in ms
task_size_kb	Incoming request size, in KB
type_cost	Numeric cost/complexity indicator for request type
replica_cores	Number of CPU cores configured for replica
replica_speed	Relative processing-speed setting
est_service_ms	Estimated service time for this request on this replica

Target: **response_ms**, the actual simulated response time in milliseconds. Features are inputs; the target is what the model learns to predict.

Why synthetic data?

We needed control over bursty workloads, monitoring delays and interference. A public trace does not automatically give the counterfactual response time for the same request on every candidate replica, nor the exact collection/availability timestamps needed for our experiment. Simulation gives control, but the results must be described as simulated—not real production measurements.

6. How the simulator creates data

The project is a **discrete-event simulation**: it moves from event to event, such as a request arriving, a monitoring record becoming available, or a request completing. It does not need to update every tiny time step.

File	Role
sa_plb/config.py	Defines replica capacities and simulation parameters.
sa_plb/workload.py	Generates request traces and monitoring schedules.
sa_plb/sim.py	Builds events and runs routing policies.
sa_plb/policies.py	Implements RR, WRR, ML routing and SA-PLB.
sa_plb/ml.py	Prepares features, trains predictor and predicts.
sa_plb/experiment.py	Runs conditions and collects results.
run_all.py	Runs the full experiment pipeline.
app.py	Streamlit dashboard for interactive demonstrations.

Workload terms

Arrival: time a request enters. **Request size:** work size in KB. **Request type:** category with a computational cost. **Peak utilisation:** workload-generation setting. **Interference:** simulated slowdown episodes that make processing less predictable.

Monitoring terms

Observation: snapshot of replica metrics such as CPU, memory, active requests, queue and recent response time. **collected_at:** when the monitor measured the replica. **available_at:** when that observation reaches the load balancer and can be used. **Latest available observation:** newest observation whose availability time is no later than the request time.

7. ML model: what and why

Our implementation uses **HistGradientBoostingRegressor** from Scikit-learn. It is a regression algorithm, so it predicts a number—in our case, response time in milliseconds.

In simple terms, gradient boosting builds a sequence of small decision trees. Each new tree tries to correct patterns in the errors made by the earlier trees. Their outputs combine into one prediction. “Histogram-based” means continuous feature values are grouped into bins to make learning more efficient.

Why this model?

Our data is tabular and mostly numerical. The target is a number, so regression is appropriate. Boosted trees can learn non-linear relationships and feature interactions—for example, queue length and CPU usage together can affect response time. HistGradientBoostingRegressor is available in Scikit-learn and is a practical model for this feature representation.

Why not CatBoost, as in the base paper?

CatBoost is also a gradient-boosted tree method and is especially convenient for datasets with categorical inputs. Our final feature representation is numerical: for example, request type is represented through a numeric feature called *type_cost*. We selected the Scikit-learn histogram gradient-boosting regressor as a practical fit for this numerical tabular dataset and implementation.

> Do not say our model is better than CatBoost. We did not conduct a controlled head-to-head experiment with both algorithms on the same data, features, split and tuning budget. Our reason is implementation fit and simplicity, not proven superiority.

Did we create our own ML model?

Yes, in the practical sense: we generated our own training examples and trained/evaluated a model for this project. No, in the algorithm-invention sense: HistGradientBoostingRegressor is an existing Scikit-learn algorithm. Our project contribution is the staleness-aware routing mechanism.

8. How we trained the model

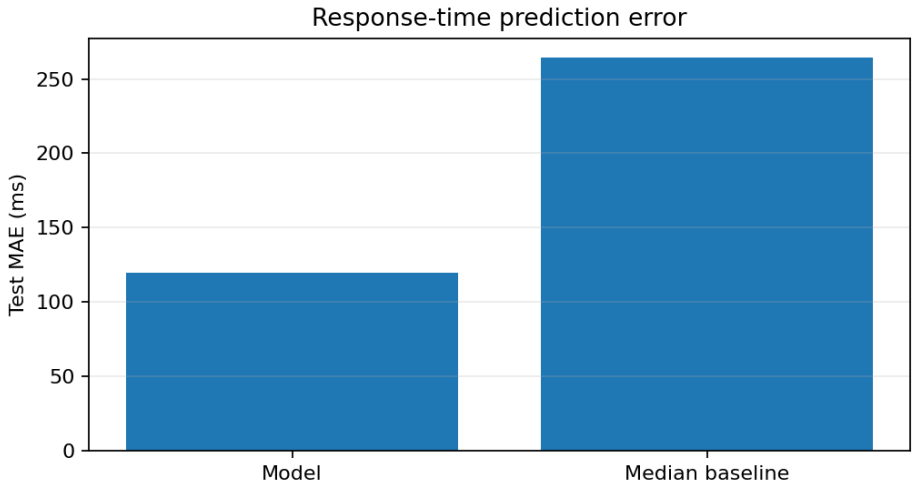
1) Generate examples in the simulator. 2) Use the ten features listed above as inputs and response_ms as target. 3) Train on log(response_ms), then exponentiate predictions back to milliseconds. 4) Split data by periods into training, validation and test. 5) Compare candidate settings on validation data. 6) Evaluate the selected model on held-out test data.

Recorded selected settings

max_iter = 100; learning_rate = 0.1; max_depth = 4.

Recorded model evaluation

Metric	Our model	Median baseline
Test MAE	119.595 ms	264.183 ms
Test RMSE	246.731 ms	602.787 ms



MAE is the average absolute difference between predicted and actual response time. An MAE of 119.6 ms means the average absolute prediction error was about 119.6 ms on this test set. **RMSE** penalises larger errors more heavily; ours was about 246.7 ms.

The model beat the median baseline on this recorded test split. However, train, validation and test response-time distributions differ; the validation period has a different workload/interference mix from the test period. This is a limitation to disclose. These scores do not establish accuracy on real cloud traffic.

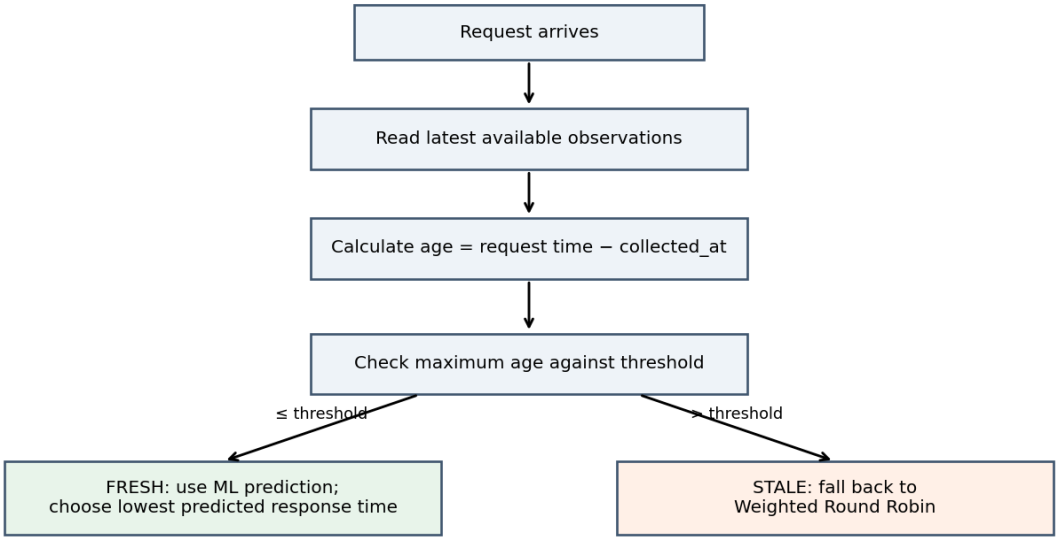
9. How did we induce staleness?

We simulated delayed monitoring updates. We did not simply edit CPU or memory values to make them old. Each monitoring observation has two timestamps: **collected_at** (when the measurement was taken) and **available_at** (when it becomes visible to the load balancer).

Example: collected_at = 5.0 s and available_at = 6.5 s means the measurement took 1.5 seconds to arrive. If a request arrives at 7.0 s, the age of that observation is 7.0 – 5.0 = 2.0 seconds.

Metric age = current request/decision time – collected_at

The recorded experiment includes monitoring delays of 0.1 s, 1.5 s and 4.0 s. Configured delay and metric age are related but not identical: actual age also depends on the time the last measurement was collected and the request arrival time.



Freshness rule in our code

We calculate the ages of the latest available observations for the candidate replicas and check **max(ages)** ≤ **threshold**. If true, the set is considered fresh and ML routing is used. If false, at least one candidate observation is too old, so SA-PLB falls back to WRR. This is a conservative rule: one stale candidate is enough to trigger fallback.

The recorded full experiment selected a threshold of **4.0 seconds** from the tested options using validation conditions. It is not a universal boundary; it is the selected value for this simulation setup.

10. One request through SA-PLB

Suppose the threshold is 1 second and the latest observation ages are: R0 = 0.7 s, R1 = 2.1 s, R2 = 1.5 s. The maximum age is 2.1 s. Since 2.1 > 1, the check fails and SA-PLB uses WRR. It does not use ML to choose the replica for this request.

If the ages were 0.7 s, 0.4 s and 0.9 s, the maximum would be 0.9 s. Since 0.9 ≤ 1, the freshness check passes and ML chooses the replica with the lowest predicted response time.

Why WRR fallback?

WRR is a capacity-weighted routing rule that does not depend on the potentially stale monitoring features used by the ML predictor. It is the chosen alternative when the predictor inputs fail the freshness check. It is not guaranteed to be optimal; it is our fallback policy.

11. Experiments and commands

The recorded full experiment tests interference levels of 0%, 10% and 25%, and monitoring delays of 0.1 s, 1.5 s and 4.0 s. Validation seeds were 100 and 101; test seeds were 0, 1 and 2. A random seed makes a random simulation repeatable. Three test seeds are a small sample, not statistical proof.

Run from the project folder

```
cd ~/Desktop/sa-plb
source .venv/bin/activate

# Full experiment pipeline
python run_all.py --out results_full

# Interactive dashboard
streamlit run app.py
```

The full experiment previously took about 10 minutes 40 seconds. The dashboard uses a shorter three-minute workload for faster interactive demonstrations. Do not rerun the full experiment unless code or settings changed.

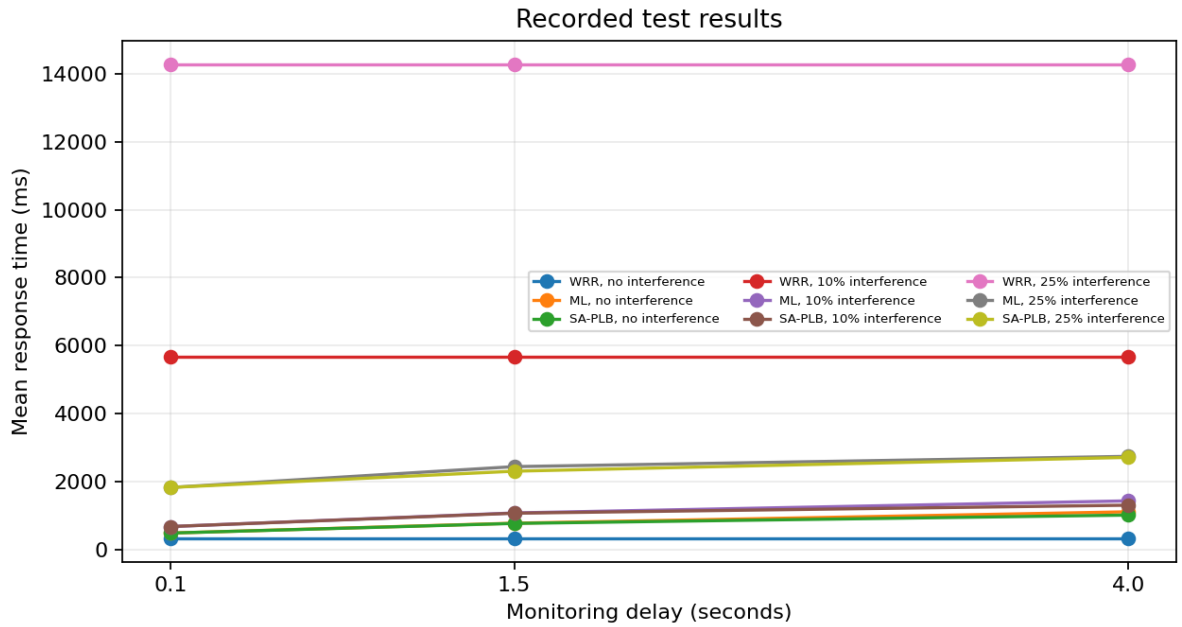
Basic Git commands

```
git status
git add README.md .gitignore app.py run_all.py sa_plb/
git commit -m "Update SA-PLB project"
git push
```

Avoid adding .venv/ and large result folders to Git unless you deliberately want to version them. The .gitignore created during setup excludes .venv, Python caches, .DS_Store, results_full and results_quick.

12. Results: what do they say?

The chart below uses the recorded mean response times averaged over test seeds. Lower is better. It plots WRR, ML and SA-PLB for each interference level and monitoring delay.



Interference	Delay	WRR ms	ML ms	SA-PLB ms
0%	0.1 s	332.63	480.44	480.44
0%	1.5 s	332.63	771.22	764.58
0%	4.0 s	332.63	1106.51	1010.37
10%	0.1 s	5679.05	672.26	672.26
10%	1.5 s	5679.05	1072.95	1065.34
10%	4.0 s	5679.05	1428.81	1301.26
25%	0.1 s	14281.70	1822.78	1822.78
25%	1.5 s	14281.70	2436.15	2301.95
25%	4.0 s	14281.70	2738.09	2708.26

Interpretation

• With 0% interference, WRR has the lowest mean response time in all three delay settings.
• With 10% and 25% interference, ML and SA-PLB have much lower means than WRR in the recorded runs.
• SA-PLB is close to ML and modestly better in several higher-delay conditions because the freshness check sometimes chooses the WRR fallback.
• At 0.1 s delay, SA-PLB and ML are identical in the listed interference conditions because fallback did not trigger in those cases.

Do not claim SA-PLB always wins. The results show a conditional trade-off: WRR is strong without interference, while predictive routing performs much better in the simulated interference conditions. SA-PLB is a guarded predictive policy, not a universal winner. The experiment uses only three test seeds and simulated assumptions.

Other metrics

p95: 95% of requests completed at or below this response time. **p99:** the 99th percentile. **Throughput:** completed requests per second. **Fallback rate:** fraction of SA-PLB decisions that used WRR. **Age mean/max:** average/maximum age of observations used at decision time.

13. Practical evaluation questions and answers

Q. What is the project solving?
A. Risk of ML routing decisions being based on outdated monitoring data.

Q. What is load balancing?
A. Distributing incoming requests among backend replicas.

Q. What is the base paper's idea?
A. Compare RR, WRR and ML latency prediction for candidate backends.

Q. Which model did the base paper use?
A. Our earlier notes identify CatBoost regression; verify the original paper before formal citation.

Q. Which model did we use?
A. HistGradientBoostingRegressor from Scikit-learn.

Q. Why did we choose it?
A. Our inputs are numerical tabular features; it is a practical gradient-boosted regressor for predicting a numeric response time.

Q. Is our model better than CatBoost?
A. We cannot claim that; we did not run a controlled head-to-head comparison.

Q. Did we invent a new ML algorithm?
A. No. We trained our own model with an existing algorithm. Our contribution is the freshness-aware routing layer.

Q. What does the model predict?
A. Response time in milliseconds for a request on a candidate replica.

Q. What is the target?
A. response_ms.

Q. Name the ten features.
A. obs_active, obs_queue, obs_cpu_pct, obs_mem_pct, obs_recent_rt_ms, task_size_kb, type_cost, replica_cores, replica_speed, est_service_ms.

Q. Why synthetic data?
A. To control bursty workload, monitoring delay and interference, and obtain simulated outcomes.

Q. Is it production data?
A. No. It is generated by our Python simulation.

Q. What is collected_at?
A. When the monitoring measurement was taken.

Q. What is available_at?
A. When the measurement becomes visible to the load balancer.

Q. How is staleness induced?
A. By delaying the availability of monitoring observations after collection.

Q. How is metric age calculated?
A. Current request time minus collected_at.

Q. Why subtract collected_at, not available_at?
A. Age measures how old the measurement itself is; available_at only controls when the system can use it.

Q. What is the freshness rule?
A. max(ages) <= threshold; every candidate observation must pass.

Q. What threshold was selected?
A. 4.0 seconds in the recorded full experiment, chosen among tested options using validation conditions.

Q. What happens when fresh?
A. Use ML and choose the candidate with the lowest predicted response time.

Q. What happens when stale?
A. Use WRR instead of the ML routing decision.

Q. Why WRR fallback?
A. It provides a capacity-weighted choice without relying on the stale monitoring inputs used by the predictor.

Q. How is the comparison fair?
A. Policies receive the same workload trace and monitoring schedule within each run.

Q. How many rows were in the final split?
A. 54,749 train; 9,962 validation; 8,700 test.

Q. Why validation data?
A. To select model settings without tuning on the final test set.

Q. What were the selected settings?
A. max_iter=100, learning_rate=0.1, max_depth=4.

Q. What was test MAE?
A. 119.595 ms, compared with 264.183 ms for the median baseline.

Q. What is MAE?
A. Average absolute difference between prediction and actual response time.

Q. What is RMSE?
A. Error metric that gives larger errors more influence; our test RMSE was 246.731 ms.

Q. Did SA-PLB beat WRR everywhere?
A. No. WRR won in no-interference conditions; SA-PLB beat WRR in the recorded interference conditions.

Q. What are key limitations?
A. Simulation only, three test seeds, different train/validation/test response distributions, and no controlled comparison against CatBoost.

Q. What would you improve?
A. More test seeds and workloads, broader threshold tuning, controlled CatBoost comparison, and validation with real telemetry if available.

14. Glossary and final revision

Term	Plain meaning
Load balancer	Chooses the backend for each incoming request.
Replica	One running copy/instance of the service.
Latency / response time	Delay until a request completes; reported in ms here.
Observation	Snapshot of measured replica metrics.
collected_at	Time the measurement was taken.
available_at	Time the load balancer can first use the observation.
Staleness / metric age	How long ago the measurement was collected at decision time.
Freshness threshold	Maximum observation age accepted by the rule.
Fallback	Alternative policy used when the main path is not trusted.
Feature	Input value supplied to the ML model.
Target	Value the model is trained to predict.
Training set	Examples used to fit the model.
Validation set	Examples used to choose settings.
Test set	Held-out examples used for evaluation.
MAE / RMSE	Measures of prediction error.
Throughput	Completed requests per unit time.
p95 / p99	95th / 99th percentile response time.
Random seed	Fixed starting value for repeatable random runs.
Interference	Simulated slowdown affecting processing.

60-second project explanation

“Our project is Staleness-Aware Predictive Load Balancing. A load balancer distributes requests across backend replicas. The base research idea compares Round Robin, Weighted Round Robin and an ML approach that predicts response latency for candidate backends. We focused on the risk that monitoring data used by the ML model may be outdated. We built a Python discrete-event simulation with three replicas and generated synthetic workloads, monitoring observations and response-time labels. We trained a Scikit-learn HistGradientBoostingRegressor to predict response time. Our SA-PLB layer calculates observation age as request time minus collected_at. If the maximum age is within the threshold, it uses ML; otherwise, it falls back to WRR. We compare RR, WRR, ML and SA-PLB on the same simulated workloads. Results show a trade-off: WRR is strong without interference, while predictive routing performs better in the simulated interference conditions. Our contribution is the freshness-aware routing gate and fallback mechanism.”

Before your evaluation: verify the base-paper model details against the original paper PDF. Keep the earlier hybrid_dataset.csv artifact separate from the final training-log split. Describe the data as simulated, and do not claim SA-PLB is universally superior.